

A formula for risk in chains of AI agents

Formula note, revision 5

Lawrence J. Genobia, independent researcher

September 29, 2026 (HST) — preprint, not peer reviewed

1 Summary

When AI agents work in a chain, one agent's error can become the next agent's input. The simple independent-step calculation understates risk whenever errors are inherited. Dependent-failure models already exist in reliability theory; this note gives a formula, exact within the stated model, for the chance of at least one high-stakes outcome in a chain of N agents, each taking n steps, where an anomaly upstream raises the anomaly rate downstream by a contamination multiplier c and contaminates the next agent's input with probability g . It includes a step-by-step method with a worked example, the matrix form, and a spreadsheet shortcut. The formula reduces to the standard independent result when $c = 1$ and when $g = 0$.

An anomaly here is any output that the implementer's written detector rule marks as wrong or non-conforming. The formula does not depend on which kinds of anomaly are counted.

Purpose. This note is written for the agentic level: chains of AI agents in which one agent's output becomes the next agent's input and agents take actions. There, a single error can carry into the next action when nothing checks it in between. The formula gives people who build, buy or rely on such chains a way to estimate the chance of at least one serious failure, and to compare ways of lowering it before a chain is built or run. It estimates and compares risk. It does not stop any error itself.

Symbol	Meaning	Who supplies it
p	Chance one step produces an anomaly, on clean input	Implementer
q	Chance one anomaly leads to a high-stakes outcome	Implementer
c	Contamination Multiplier: how much p rises when the previous agent's output contained an anomaly	Implementer
g	Handoff probability: given that the previous agent produced at least one anomaly, the chance that the next agent's input is contaminated. At $g = 1$ it always is (the default). At $g = 0$ it never is, and the chain is independent.	Implementer
n	Steps per agent	Implementer
N	Agents in the chain	Implementer

Values used in worked examples are placeholders, not rate claims.

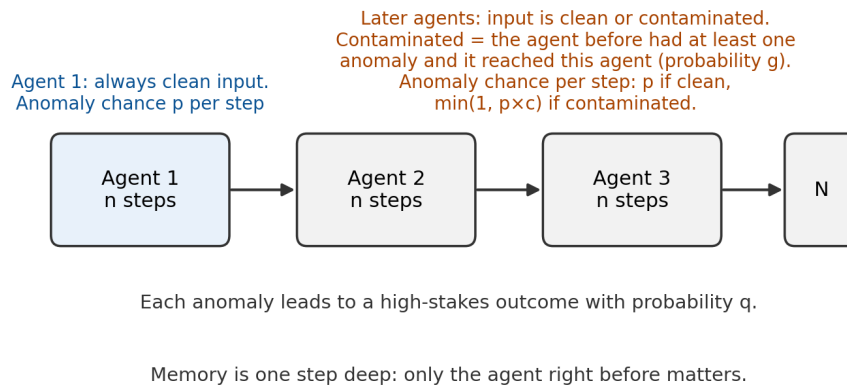
2 The model

The formula is exact for the model below. Each item is a stated scope choice, not an error, and each can be improved (the theory paper, to be published separately, lists them as open slots).

- A chain of N agents runs in order. Each agent performs n steps.
- Agent 1's input is clean.

- An agent's input is contaminated with probability g when the previous agent produced at least one anomaly. At $g = 1$, that always happens. At $g = 0$, it never does and agents are independent.
- On clean input, each step produces an anomaly with probability p . On contaminated input, the probability is $\min(1, p \times c)$.
- Each anomaly independently leads to a high-stakes outcome with probability q .
- H means at least one high-stakes outcome anywhere in the chain.
- Memory is one step deep, and c is fixed.

Allowed ranges: $0 \leq p \leq 1$, $0 \leq q \leq 1$, $c \geq 0$, $0 \leq g \leq 1$, $n \geq 1$, $N \geq 1$.



Result: $P(H)$ = chance of at least one high-stakes outcome in the chain

Figure 1. The chain model. Agent 1 always receives clean input. Each later agent receives contaminated input when the agent before it produced at least one anomaly and that anomaly reached it (probability g).

3 The formula

Three layers: a plain step-by-step method with a worked example (3.1), the technical form (3.2), and the verification (3.3).

3.1 Layer 1: step by step, with a worked example

For each agent, work out three chances given its input state: no anomaly, anomalies but no high-stakes outcome, and a high-stakes outcome. Then carry two running numbers forward: the chance of no high-stakes outcome so far with the next agent clean, and with the next agent contaminated. Repeat for each agent.

Example inputs: $p = 0.4$, $q = 0.5$, $c = 5$, $n = 3$, $N = 2$, $g = 1$. Because $p \times c = 2$ is above 1, the cap applies: $r = \min(1, 0.4 \times 5) = 1$.

1. **Agent 1 is clean.** No anomaly: $0.6^3 = 0.216$. No high-stakes outcome at all: $0.8^3 = 0.512$. So anomalies but no high-stakes outcome: $0.512 - 0.216 = 0.296$. A high-stakes outcome: $1 - 0.512 = 0.488$.
2. **Agent 2 depends on agent 1.** After no anomaly (0.216), it is clean and has a 0.512 chance of no high-stakes outcome. After an anomaly without high stakes (0.296), it is contaminated ($g = 1$) and has a $(1 - 1 \times 0.5)^3 = 0.125$ chance.

3. **Chance of no high-stakes outcome in the chain:** $0.216 \times 0.512 + 0.296 \times 0.125 = 0.110592 + 0.037 = 0.147592$.

4. **$P(H) = 1 - 0.147592 = 0.852408$.**

If the steps were independent, the answer would be $1 - 0.8^6 = 0.737856$. Contamination lifts the risk from about 73.8% to about 85.2%. Wolfram Alpha confirmed 0.852408 (Sept 28, 2026, 1:21 PM HST). Without the cap, the same calculation gives a wrong 0.889408.

3.2 Layer 2: technical form

$$r = \min(1, p \times c)$$

$$a_c = (1-p)^n$$

$$a_D = (1-r)^n$$

$$s_c = (1-pq)^n$$

$$s_D = (1-rq)^n$$

$$M_g = \begin{bmatrix} a_c + (s_c - a_c)(1-g) & a_D + (s_D - a_D)(1-g) \\ (s_c - a_c) \times g & (s_D - a_D) \times g \end{bmatrix}$$

$$P(H) = 1 - [1, 1] \cdot M_g^N \cdot [1, 0]^T$$

Subscript C means clean input and D means contaminated input. The letter a is the chance of no anomaly, and s is the chance of no high-stakes outcome.

For two agents:

$$P(H) = 1 - \{s_c \times [a_c + (s_c - a_c)(1-g)] + s_D \times (s_c - a_c) \times g\}$$

At $g = 1$ this is $1 - [a_c \times s_c + (s_c - a_c) \times s_D]$. At $c = 1$ and $g = 1$ it collapses to $1 - (1-pq)^{2n}$.

Spreadsheet shortcut, with no matrices:

$$S_0 = 1, \quad S_1 = s_c, \quad \text{and for } k \geq 2: \quad S_k = \tau \times S_{k-1} - \Delta \times S_{k-2}$$

$$\tau = a_c + (s_c - a_c)(1-g) + (s_D - a_D) \times g$$

$$\Delta = g \times (a_c \times s_D - a_D \times s_c)$$

$$P(H) = 1 - S_N$$

Status: matched the matrix form to 15 decimals on 20,000 random inputs in the author's own code (Sept 28), and matched again to within $1e-9$ on a fresh 20,000-input check (Sept 29, AI-assisted code, not independent). Not yet independently reproduced.

3.3 Layer 3: verification

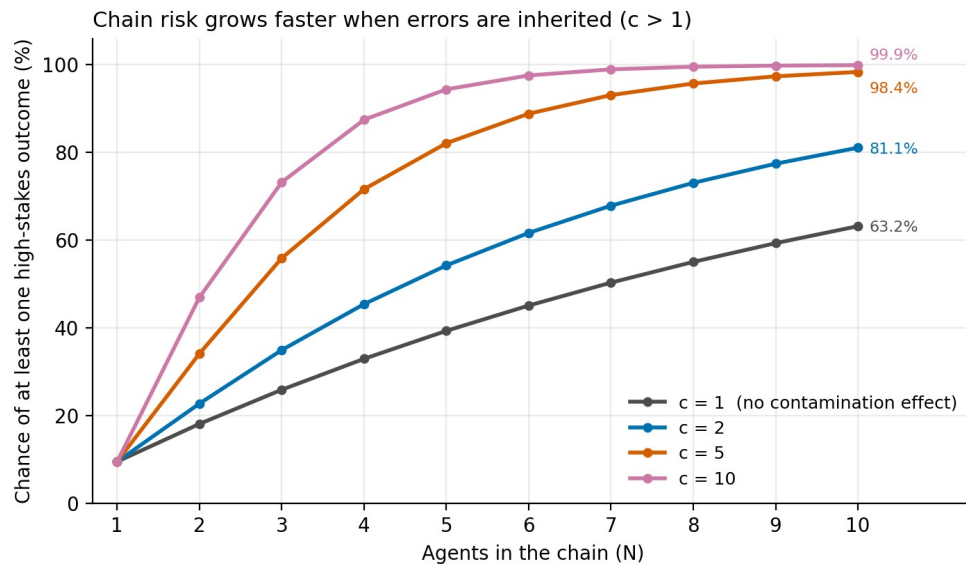
Eight Wolfram Alpha checks (six prior; two new with g strictly between 0 and 1, September 28, 2026), exact enumeration of every outcome in 22 small cases, a 3-million-trial simulation, and blind rebuilds of the formula by Gemini, ChatGPT and Copilot. On September 29, 2026 the worked example, the N-agent table, the per-agent rates and the Test D planning numbers were recomputed and all matched. The full verification record will accompany the theory paper, to be published separately.

The two Wolfram Alpha checks with g strictly between 0 and 1 used the inputs below. The formula in Section 3.2 reproduces both values (AI-assisted code, September 29, 2026; not an independent check).

Check	Inputs	P(H)
$g = 0.5$	$p = 0.05, q = 0.3, c = 3, g = 0.5, n = 8, N = 4$	0.43792
$g = 0.3$, cap active	$p = 0.4, q = 0.5, c = 5, g = 0.3, n = 3, N = 2$ (cap: $r = 1$)	0.77222

4 What the formula shows

Five results follow from the model. Each is labeled by how far it has been established.



Example inputs only: $p = 0.001, q = 0.1, n = 1,000$ steps per agent, $g = 1$. Not measured rates.

Figure 2. Chain risk against chain length for four values of the contamination multiplier c (example inputs only: $p = 0.001, q = 0.1, n = 1,000, g = 1$). At $c = 1$ the curve is the standard independent result.

Result	Plain meaning	Status
At $c = 1$: $P(H) = 1 - (1 - pq)^{nN}$	No contamination effect. This is the standard independent result.	Proved by algebra. Confirmed by Gemini and Wolfram Alpha.
When $c \geq 1$: independent result $\leq P(H) \leq 1 - (1 - rq)^{nN}$	Contamination only adds risk, and never more than if every agent were contaminated. When $c \leq 1$, the inequalities reverse and the independent result is the ceiling.	Proved (Section 4.1; the author's derivation with AI assistance, not yet independently checked). Also tested: 20,000 random inputs (c from 0 to 50, g free) on September 28, 2026 and again on September 29, 2026, zero violations.

Result	Plain meaning	Status
When $c \geq 1$, $P(H)$ never falls as N , c , p , q or n rises	More agents never make the chain safer. More contamination or more steps never do either.	For N : proved, because the chance of no high-stakes outcome can only fall as agents are added. For c , p , q and n : tested on 30,000 random inputs (September 28, 2026; g free; c from 0 to 50) and 20,000 more on September 29, 2026, zero violations. Not proved.
$P(\text{no } H) \leq \max(s_c, s_D)^N$	If both are below 1, risk heads toward 100% as the chain grows.	Proved: each agent multiplies the surviving total by at most $\max(s_c, s_D)$. Also tested on 30,000 inputs.
Given no high-stakes outcome so far, the chance of one at each added agent approaches $1-\lambda$, where λ is the larger eigenvalue of M_g	A long-run per-agent rate for the survivors. The extra cumulative risk from one more agent shrinks toward zero, because fewer and fewer chains are still unharmed.	Standard matrix theory (Darroch and Seneta 1965; Seneta 2006; Froyland and Stuart 2016): $P(\text{no } H \text{ after } N)$ behaves like $K\lambda^N$ for large N when the matrix is not degenerate. Checked September 29, 2026 with AI-assisted code on the example inputs below (ratio of successive survival probabilities matches λ); not checked on Wolfram Alpha.

Example inputs only ($p = 0.001$, $q = 0.1$, $n = 1000$). The long-run chance of harm at the next agent, given none so far ($1-\lambda$), is 9.5% at $c = 1$, 16.2% at $c = 2$, 38.6% at $c = 5$ and 62.8% at $c = 10$. Chain risk at $N = 10$ is 63.21% at $c = 1$ and 98.39% at $c = 5$. At $N = 2$ and $c = 5$ it is 34.15%. Wolfram Alpha confirmed the $N = 10$ and $N = 2$ values.

Mathematical background. The calculation is a standard absorbing-chain construction. The matrix M_g holds the probabilities of moving between the two unharmed states, and its N th power gives the probability of being unharmed after N agents (Grinstead and Snell 1997, Section 11.2). The long-run behavior is governed by the largest eigenvalue of that matrix, which is the classical setting of quasi-stationary distributions (Darroch and Seneta 1965; Seneta 2006; Froyland and Stuart 2016). None of these sources says anything about AI agents or about whether c is above 1.

4.1 Proof of the bounds and of the certainty limit

Let $s_c = (1-pq)^n$ and $s_D = (1-rq)^n$ be the chances that an agent has no high-stakes outcome when its input is clean and when it is contaminated. When $c \geq 1$, $r \geq p$, so $s_D \leq s_c$.

Whatever happened earlier in the chain, the chance that the next agent has no high-stakes outcome depends only on whether its input is clean or contaminated. Given that the chain is unharmed so far, that chance is s_c , s_D , or an average of the two. So each agent multiplies the surviving probability by a number between s_D and s_c . After N agents:

$$s_D^N \leq P(\text{no } H) \leq s_c^N$$

Subtracting from 1 gives $1-s_c^N \leq P(H) \leq 1-s_D^N$, and $1-s_c^N = 1-(1-pq)^{nN}$, the independent result. When $c \leq 1$ the inequalities reverse.

If both s_c and s_D are below 1, then $P(\text{no } H) \leq \max(s_c, s_D)^N$, which tends to 0, so the risk approaches 100%.

Status: this derivation is the author's, with AI assistance. It has not been independently checked. A separate numerical check on September 29, 2026 confirmed, on 129,554 agent-by-agent factors from random inputs, that every factor fell between s_D and s_c . The proof does not cover monotonicity in c , p , q and n , which stays "tested, not proved".

5 Using the formula

Values used in this section are placeholders ($p = 0.001$, $q = 0.1$, $n = 1,000$, $g = 1$), not rate claims. Every number below was computed with AI-assisted code, in both the matrix form and the spreadsheet shortcut of Section 3.2, and is not independently reproduced.

5.1 What to do with the result

The result is one number for one chain, as described by its inputs. To compare designs, change one input, run the calculation again and compare the two results. Each input points to something a designer can change. These are examples of what to try, not findings:

- **p**: fewer anomalies per step (a clearer task, narrower instructions, a different model).
- **q**: less damage per anomaly (limited permissions, reversible actions, a person signing off on high-stakes steps).
- **c**: less contamination (hand the next agent verified facts, not the raw upstream output).
- **g**: checks between agents, so fewer anomalies reach the next one.
- **n and N**: fewer steps and shorter chains.

Section 4 shows that the result does not fall as N , c , p , q or n rises (proved for N ; tested, not proved for the others). The formula does not say which change is cheapest or best for a particular system. The implementer measures that.

5.2 When the inputs are hard to measure

Some inputs, especially c and q , are hard to measure. The formula can still be used in four ways.

1. **A floor from p and q alone.** The independent result, $1 - (1-pq)^{nN}$, needs only p , q , n and N . When $c \geq 1$ it is a floor: the chain can be no safer than this (proved in Section 4.1). When $c \leq 1$ the inequalities reverse and it is the ceiling. With the placeholder inputs and $N = 3$ agents, the floor is 25.92%.
2. **A range for c , not one measured value.** Run the formula across a plausible range and report all of it, as Figure 2 does. The same approach works for any input that is hard to measure. Table 1 gives the placeholder example.
3. **A threshold for c .** Instead of measuring c , choose a risk limit and find the value of c at which the chain reaches it (a tipping point). Then judge whether the chain's c is above or below that value. Because the result does not fall as c rises (Section 4; tested, not proved), a chain whose c is below the threshold stays below the limit. Table 2 gives the placeholder example for a 50% limit.
4. **The default $g = 1$.** When g is unknown, set $g = 1$, which assumes every anomaly reaches the next agent, and say so. This is the default used throughout this note.

Table 1. Chain risk (%) by contamination multiplier c and chain length N (placeholder inputs).

N (agents)	c = 1	c = 2	c = 5	c = 10
2	18.13	22.75	34.15	46.98
3	25.92	34.97	55.93	73.24
5	39.35	54.27	82.13	94.42

Table 2. Value of c at which chain risk reaches 50% (placeholder inputs).

N (agents)	Independent floor ($c = 1$), %	c at which risk reaches 50%
2	18.13	11.66
3	25.92	3.97
5	39.35	1.71

A longer chain reaches the limit at a smaller c . This is the reading of Table 2 for the placeholder inputs, not a statement about any real system.

5.3 What a check cannot cover

A check between agents can only stop an anomaly that the written detector rule can see. Anomalies the rule cannot see pass to the next agent unchanged, so a chain with checks still carries risk from those kinds. A check lowers g only for the kinds it can detect. Report the kinds a check cannot see alongside the result. This is the limit on what the calculation can promise: it shows where a check would matter in the model, and it does not make a chain with unchecked kinds safe.

6 What this note does not do

- **It gives no rates.** Every output number depends on p , q , c and g , which the user measures from their own system. Values in examples are placeholders.
- **It does not show that contamination raises risk.** That is the premise (c above 1). The formula shows what follows if it holds. The theory paper, to be published separately, states six pre-registered results that would count against the premise.
- **It does not define anomaly kinds.** The author's own labels and definitions will be published separately and are not needed to apply the formula.
- **It does not claim the model fits real chains.** The model treats contamination as on or off (one anomaly and a hundred anomalies have the same effect), memory as one step deep, and anomalies as leading to high-stakes outcomes independently. Real chains may differ on each point.
- **It is not peer reviewed.** The arithmetic is verified as stated in Section 3.3. The bounds, the certainty limit and monotonicity in N are proved in Section 4.1 (the author's derivation with AI assistance, not independently checked). Monotonicity in c , p , q and n is tested, not proved.

7 Reproducibility

Section 3 specifies the formula in three forms. The program below (Python 3.9 or later, numpy only) reproduces the worked example and the table in Section 4. Its output was checked on September 29, 2026.

```
# Minimal reproduction: formula, worked example, N-agent table.
# Python 3.9 or later; numpy only.
import numpy as np

def p_high_stakes(p, q, c, g, n, N):
    """Chance of at least one high-stakes outcome in a chain of N agents,
    each taking n steps."""
    r = min(1.0, p * c) # anomaly rate on contaminated input
    aC, sC = (1 - p) ** n, (1 - p * q) ** n # clean: no anomaly / no harm
    aD, sD = (1 - r) ** n, (1 - r * q) ** n # contaminated input
    M = np.array([[aC + (sC - aC) * (1 - g), aD + (sD - aD) * (1 - g)],
                  [(sC - aC) * g, (sD - aD) * g]])
    return 1 - np.linalg.matrix_power(M, N) @ np.array([1, 0])

print("Worked example:", round(p_high_stakes(0.4, 0.5, 5, 1, 3, 2), 6))
print("Independent   :", round(1 - 0.8 ** 6, 6))
print("N-agent table (p=0.001, q=0.1, n=1000, g=1), percent:")
for N in (1, 2, 3, 5, 10):
    row = [float(round(100 * p_high_stakes(0.001, 0.1, c, 1, 1000, N), 2))
            for c in (1, 2, 5, 10)]
    print(N, row)
```

Expected output:

```
Worked example: 0.852408
Independent   : 0.737856
N-agent table (p=0.001, q=0.1, n=1000, g=1), percent:
1 [9.52, 9.52, 9.52, 9.52]
2 [18.13, 22.75, 34.15, 46.98]
3 [25.92, 34.97, 55.93, 73.24]
5 [39.35, 54.27, 82.13, 94.42]
10 [63.21, 81.11, 98.39, 99.93]
```

8 References

- Darroch, J. N., and Seneta, E. (1965). On quasi-stationary distributions in absorbing discrete-time finite Markov chains. *Journal of Applied Probability* 2(1), 88–100.
- Froyland, G., and Stuart, R. M. (2016). Cheeger inequalities for absorbing Markov chains. *Advances in Applied Probability* 48(3), 631–647. <https://doi.org/10.1017/apr.2016.20>
- Grinstead, C. M., and Snell, J. L. (1997). *Introduction to Probability*, 2nd ed. American Mathematical Society, Section 11.2.
- Seneta, E. (2006). *Non-negative Matrices and Markov Chains*, revised printing. Springer.

9 Disclosure, licence and how to cite

Authorship and AI assistance. The formula, its name, and the meaning of each variable are the author's. AI assistance (Claude, Anthropic) was used for computation, structure and wording at the author's direction. Checks marked "author's code" or "AI-assisted code" are not independent checks.

Licence. Creative Commons Attribution 4.0 International (CC BY 4.0).

How to cite. Genobia, L. J. (2026). *A formula for risk in chains of AI agents* (Formula note, revision 5). Preprint. A DOI is assigned when the record is deposited.

Revision note. Revision 5 (September 29, 2026, HST). Changes from revision 4: a Purpose paragraph added to Section 1; new Section 5, Using the formula (what to do with the result, four ways to use the formula when inputs are hard to measure, and what a check cannot cover), with two placeholder tables; later sections renumbered 6 to 9. No formula, earlier computed value, statistic or claim of proof was changed.